

Vivek Notes For Creating Peering In VPC and Connecting Two Different Instances To another (Public & Private)

VPC-1

192.168.0.0/16

Public Subnet

10.10.1.0/24

Public EC2

|

VPC PEERING

|

VPC-2

192.178.0.0/16

Private Subnet

10.20.1.0/24

Private EC2

Step 1 — Create VPC-1

Go to:

AWS Console → VPC → Your VPCs → Create VPC

Name VPC-1-Public

IPv4 CIDR 192.168.0.0/16

Step 2 — Create Public Subnet in VPC-1

Go to:

VPC → Subnets → Create subnet

Select VPC: VPC-1-Public

Subnet name: Public-Subnet-1

Availability Zone: ap-south-1a

IPv4 CIDR: 192.168.1.0/24

Step 3 — Create VPC-2

Create another VPC:

Name: VPC-2-Private
CIDR: 192.178.0.0/16

⚠ **Very important:** VPC-2 must have a different CIDR from VPC

- 1) Always Choose Different CIDR IP while making VPC's

Step 4 — Create Private Subnet in VPC-2

Go to:

VPC → Subnets → Create subnet

VPC: VPC-2-Private Subnet

name: Private-Subnet-2

Availability Zone: ap-south-1a

IPv4 CIDR: 192.178.10.0/24

NOW We Have :-

VPC-1

192.168.0.0/16

└─ Public Subnet

192.168.1.0/24

VPC-2

192.178.0.0/16

└─ Private Subnet

192.178.10.0/24

Step 5 — Create Internet Gateway for VPC-1

Go to:

VPC → Internet Gateways → Create internet gateway

Name:

Net-VPC1

Then:

Actions → Attach to VPC

Select:

VPC-1-Public

Step 6 — Route Table for Public Subnet

Go to:

VPC → Route Tables → Create route table

Use:

Name: Public-Route-VPC1

Select VPC :- VPC-1-Public

Go to:

Routes → Edit routes → Add route

Add:

Destination: 0.0.0.0/0

Target: Internet Gateway

Net-VPC1

Then:

Subnet associations → Edit subnet associations

Select:

Public-Subnet-1

Step 7 — Launch Public EC2

Create an Ubuntu EC2 inside:

VPC-1-Public

Subnet:

Public-Subnet-1

Enable:

Auto-assign Public IP → Enable

Step 8 — Launch Private EC2

Create another Ubuntu EC2.

Select:

VPC: VPC-2-Private

Subnet: Private-Subnet-2

Auto-assign Public IP: Disable

This instance **cannot be directly accessed from the Internet.**

Step 9 — Create VPC Peering Connection

Go to:

VPC → Peering connections → Create peering connection

Use:

Name: VPC1-to-VPC2-Peering

Requester VPC:

VPC-1-Public

Accepter VPC:

VPC-2-Private

Then click:

Create peering connection

Step 10 — Accept the Peering Connection

Go to:

VPC → Peering connections

Status: Pending Acceptance

Select it.

Click On Right Side in Top Of Corner:

Actions → Accept request

Now status should become:

Active

Step 11 — Add Route in VPC-1

Go to the route table associated with:

Public-Subnet-1

Step 12 — Add Route in VPC-2

Now go to the route table associated with:

Private-Subnet-2

You need **both routes**.

VPC-1 Route Table

192.168.0.0/16 → Peering

↑↓

VPC-2 Route Table

192.178.0.0/16 → Peering

Step 13 — Configure Security Group of Private EC2

This is another very important step.

Go to:

EC2 → Private EC2 → Security → Security Groups

Add inbound rule:

Type: All Traffic

Port: 22

Source:

0.0.0.0/0

For testing, you can alternatively use

Step 14 — Configure Security Group of Public EC2

For your public EC2, allow SSH from your own IP:

Step 15 — Copy the Private Key to Public EC2

into your public EC2 from your laptop:

```
ssh -i your-key.pem ubuntu@PUBLIC_IP
```

Note:- The Public IP is available In your VPC-1

Once you're inside the public EC2, you need the private EC2's key.

For example:

```
EC2-VPC-1private.pem
```

Step 16 — Fix Key Permissions

On the public EC2:

```
chmod 400 "Enter Your key Name"
```

Check:

ls -ltr

You should see restrictive permissions.

Step 17 —Public EC2 to Private EC2

Suppose your private EC2 has:

Private IP = 10.20.1.10

Run:

```
ssh -i EC2-VPC-1private.pem ubuntu@10.20.1.10
```

⚠ Notice there is **NO space** after ubuntu@

Your Laptop

|

Internet

|

Public EC2

VPC-1

192.168.0.0/16

|

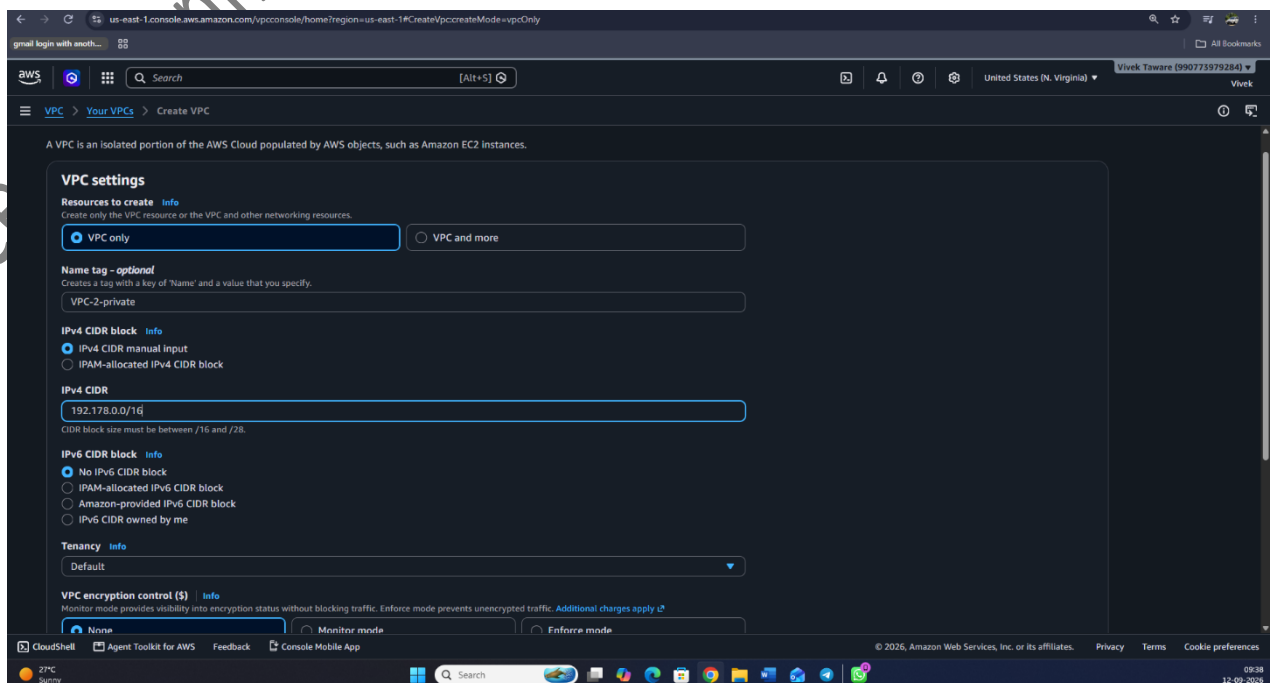
VPC Peering

|

Private EC2

VPC-2

192.78.0.0/16



us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#subnets:subnetId=subnet-0507f88d3e79df2c8

gmail login with another account

aws [Search] [Alt+S]

United States (N. Virginia) Vivik Taware (990773979284)

VPC > Subnets

VPC dashboard

Filter by VPC

AWS Global View

Virtual private cloud

- Your VPCs
- Subnets**
- Route tables
- Internet gateways
- Egress-only internet gateways
- Carrier gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peering connections
- Route servers

Security

- Network ACLs
- Security groups

PrivateLink and Lattice

- Getting started

You have successfully created 1 subnet: subnet-0507f88d3e79df2c8

Subnets (1) Info

Find subnets by attribute or tag

Subnet ID: subnet-0507f88d3e79df2c8 Clear filters

Last updated less than a minute ago Actions Create subnet

☐	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 C
☐	Subnet-1-public	subnet-0507f88d3e79df2c8	Available	vpc-0617f964ec1dd5aed VPC-1-Public	Off	192.168.1.0/24	-

Select a subnet

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

27°C Sunny 09:39 12-09-2026

aws [Search] [Alt+S]

United States (N. Virginia)

VPC > NAT gateways > Create NAT gateway

Create NAT gateway Info

A highly available, managed Network Address Translation (NAT) service that instances in private subnets can use to connect to services in other VPCs, on-premises networks, or the internet.

NAT gateway settings

Name - optional

Create a tag with a key of 'Name' and a value that you specify.

my-nat-gateway-01

The name can be up to 256 characters long.

Availability mode Info

Choose whether to deploy across all zones in the region or restrict to a single availability zone.

☒ **Regional - new**

Scales automatically across all regional AZs, simplifying management for multi AZ deployments.

☐ **Zonal**

Provides granular control within a specific availability zone, adhering to subnet level settings.

VPC

Select a VPC in which to create the regional NAT gateway.

Select a VPC

Q

vpc-0e89bcd3724652132 (default)

Public Private

Method of Elastic IP (EIP) allocation Info

Choose how IP addresses are associated with NAT gateways.

☒ **Automatic**

AWS automatically manages EIPs and AZ coverage for NAT gateways. This ensures easy scaling-- adding AZs automatically allocates EIPs, simplifying management.

☐ **Manual**

Manually assigns specific IP addresses for compliance or whitelisting. Note: Requires manual scaling to new AZs as workloads expand.

